

(I) Pumping Lemma.

Let $L \subseteq \Sigma^*$ be a regular language. $\exists p \in \mathbb{N}$ (pumping length) such that $\forall s \in L$ (s is a string) with $|s| \geq p$ can be written as :

$$s = xyz \quad \text{with } x, y, z \in \Sigma^*$$

where x, y, z are :

$$|xy| \leq p$$

$$|y| \geq 1$$

$$\text{and } \forall i \geq 0, x(y)^i z \in L$$

Here, y is the pumpable part.

Intuition: For long strings, they basically need to go through, some loop in an FA. This looping part can be repeated any amount of times without leaving the language.

Assumption:

Let L be a regular language. recognized by a DFA

$$M = (Q, \Sigma, q_0, F, \delta)$$

let $j = |Q| = \text{no. of states in } M.$

proof:

Pigeonhole Lemma (I)

Consider any $s \in L$ with $|s| \geq p$

Let $S = a_1 a_2 a_3 \dots a_m$

with $m \geq p$ and $a_i \in \Sigma^*$

Running M on S produces the state sequences as follows:

q_0

$q_1 = \delta(q_0, a_1)$

$q_2 = \delta(q_1, a_2)$

$q_3 = \delta(q_2, a_3)$

$\vdots$

$q_m = \delta(q_{m-1}, a_m)$

Since there are p distinct states in M ,

by pigeon hole principle, we can say that some states will repeat.

Say among first $p+1$ states $q_0, q_1, \dots, q_p$ some states repeat.

let $0 \leq r < t \leq p$ where

$q_r = q_t$

and $t-r \geq 1$ (they are not on the same state)

Now, let us define

$$x = a_1 a_2 \dots a_r$$

$$y = a_{r+1} a_{r+2} \dots a_t$$

$$z = a_{t+1} a_{t+2} \dots a_m$$

then $s = x.y.z$, where $|y| = t - r \geq 1$

and $|xy| = t \leq p$

Since $q_r = q_r$, these states loop back

Hence reading y from q_r returns q_r

$$\delta(q_r, y) = q_t = q_r$$

Hence for any $i \in \mathbb{N}$

$$\delta(q_r, y^i) = q_r$$

$$\Rightarrow \delta(q_0, xy^i) = \delta(q_r, y^i) = q_r$$

$$\Rightarrow \delta(q_0, xy^i z) = \delta(q_r, z) = q_m$$

Because $s \in L$ and M accepts s ,

we have $q_m \in F$

$\therefore xy^i z$ is also accepted $\forall i \in \mathbb{N}$

→ DFA-Definition:

- A finite automaton or finite state machines (FA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:
 - $Q \rightarrow$ finite set of states
 - $\Sigma \rightarrow$ finite alphabet of input symbols
 - $\delta \rightarrow$ function from $Q \times \Sigma \rightarrow Q$ (transition function)
 - $q_0 \rightarrow$ belongs to Q , is the initial state
 - $F \rightarrow F \subseteq Q$ (set of accepting / final states)

NFA-Definition:

- It is a 5 tuple: $M = (Q, \Sigma, q_0, F, \delta)$, where:
 - Q and Σ are non-empty finite sets
 - $q_0 \in Q$
 - $F \subseteq Q$ and
 - $\delta: Q \times \Sigma \rightarrow 2^Q$

E-NFA-Definition:

A non-deterministic finite automata with ϵ (λ) transitions, abbreviated as ENFA is a 5 tuple $(Q, \Sigma, q_0, F, \delta)$ where:
 Q and Σ are finite states, $q_0 \in Q$ and $F \subseteq Q$ and
 $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$

Non-Recursive Definition S^* of E-NFA:

For an E-NFA, $M = (Q, \Sigma, q_0, F, \delta)$

states $p, q \in Q$ and a string $x = a_1 a_2 \dots a_n \in \Sigma^+$, we will say M moves from p to q by a sequence of transitions, corresponding to x , if there exists an integer $m \geq n$, a sequence $b_1 b_2 \dots b_m \in \Sigma \cup \{\epsilon\}$

satisfying $b_1 b_2 \dots b_m = x$ and the sequence of states $p = p_0, p_1, \dots, p_n = q$ so that for each i with $1 \leq i \leq n$,

$$p_i \in \delta(p_{i-1}, b_i)$$

for $x \in \Sigma^+$ and $p \in Q$

$S^*(p, x)$ is a set of all states $q \in Q$, such that there is a sequence of transitions corresponding to x by which M moves from p to q .

ϵ -Closure - Definition:

Let $M = (Q, \Sigma, q_0, F, \delta)$ is an E-NFA and let s be any subset of Q , Then ϵ -closure of s is the set $\epsilon(s)$ defined as follows:

- (i) Every element of S is an element of $\epsilon(S)$
- (ii) for any $q \in \epsilon(S)$, every element of $\delta(q, \epsilon)$ is in $\epsilon(S)$
- (iii) No other element of Q are in $\epsilon(S)$

Recursive Definition of δ^* for an E-NFA:

let $M = (Q, \Sigma, q_0, F, \delta)$ be an E-NFA, the extended transition function:

$\delta^* : Q \times \Sigma^* \cup \{\epsilon\} \rightarrow 2^Q$ is defined as

$$(1) \quad q \in Q, \quad \boxed{\delta^*(q, \epsilon) = \epsilon(\{q\})}$$

(2) for $q \in Q$, $y \in \Sigma^*$ and $a \in \Sigma$, then.

$$\boxed{\delta^*(q, ya) = \epsilon \left[\bigcup_{r \in \delta^*(q, y)} \delta(r, a) \right]}$$

$\hookrightarrow \epsilon$ -closure of an ϵ -closure.

(3) A string is accepted by M , if,

$$\boxed{\delta^*(q_0, x) \cap F \neq \emptyset}$$

(4) The language recognized by M of all strings accepted by M .

Regular Expressions

Let Σ be an alphabet, then:

1) ϕ, ϵ & $a \in \Sigma$ are all regular expressions, generating the regular language $\{\}$, $\{\epsilon\}$, and $\{a\}$ respectively.

2) If r_1 and r_2 are regular expressions, then $r_1 + r_2$, $r_1 \cdot r_2$, r^* , $(r_1)^-$ are regular expressions such that

$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$

$$L(r_1 \cdot r_2) = L(r_1) \cdot L(r_2)$$

$$L(r_1^*) = (L(r_1))^*$$

$$L((r_1)^-) = L(r_1)^-$$

3) Only those derivable from the above rules are regular expressions and others are not.

Algebraic Laws (Identities) for Regular Expressions:

- (i) $\phi + R = R$
- (ii) $\phi \cdot R = R \cdot \phi = \phi$
- (iii) $\epsilon \cdot R = R \cdot \epsilon = R$
- (iv) $\epsilon^* = \epsilon$, $\phi^* = \epsilon$
- (v) $R + R = R$
- (vi) $R^* R^* = R^*$
- (vii) $RR^* = R^*R = R^+$
- (viii) $(R^*)^* = R^*$
- (ix) $\epsilon + RR^* = R^* = \epsilon + R^*R = R^*$
- (x) $(PQ)^*P = P(QP)^*$
- (xi) $(P+Q)^* = (P^*Q^*)^* = (P^*+Q^*)^*$
- (xii) $(P+Q)R = PR + QR$
 $R(P+Q) = RP + RQ$

Closure Properties of Regular Languages:

Consider a set of regular Languages, $S = \{L_1, L_2, \dots\}$

A formal language is said to be a regular language if it is accepted by DFA, NFA or ϵ -NFA or generated by regular expression or regular grammar.

Properties:

(1) If L and M are regular languages, then so are $L \cup M$, $L \cdot M$, L^*

proof

let r and s be the regular exp such that

$$L = L(r)$$

$$M = L(s)$$

$$\text{then } L \cup M = L(r+s)$$

$$L \cdot M = L(r \cdot s)$$

$$L^* = L(r^*)$$

the regular exp for $L \cup M$, $L \cdot M$ and L^* are derived from the regular exp r and s , hence this completes the proof.

(2) If L is a regular language over a given Σ , then so is L' , such that $L' = \Sigma^* - L$, i.e. the set of all regular languages are closed under complementation.

proof

let $L = L(D)$, for some DFA $D = (Q, \Sigma, q_0, \delta, F)$ then

$L' = L(E)$, where E is a DFA such that

$$E = (Q, \Sigma, q_0, \delta, Q - F)$$

i.e. E is same as D , where the final and non-final states are swapped.

③ The set of regular languages are closed under intersection and difference

proof:

from properties ① and ② and Demorgan's law, we have the following proof,

Let L_1 and L_2 be languages, then $L_1' \& L_2'$ as well $L_1 \cup L_2$ and $L_1' \cup L_2'$ are regular languages as per ① and ②.

Hence $L_1 \cap L_2 = (L_1' \cup L_2')'$ and

$L_1 - L_2 = L_1 \cap (L_2')'$ are also regular languages.

Hence proved.

Homomorphism:

let $L_1 \subseteq \Sigma^*$, $L_2 \subseteq \gamma^*$

let $H: \Sigma^* \rightarrow \gamma^*$

H is homomorphism if:

$$H(\epsilon) = \epsilon$$

$$H(xy) = H(x)H(y)$$

→ concatenated string.

Prove that for every ϵ -NFA there exists an equivalent DFA Σ^* accepting the same language.

The proof that for every ϵ -NFA M there exists an equivalent DFA M' relies on the Subset Construction method, extended to handle ϵ transitions via the ϵ closure.

Proof:

Let the ϵ -NFA be $M = (Q, \Sigma, q_0, \delta, F)$. We construct the equivalent DFA $M' = (Q', \Sigma, \delta', q_0', F')$ as follows:

- the ϵ -closure, $E(S)$, of a set of states $S \subseteq Q$ is the set of all states reachable from any state in S by zero or more ϵ transitions.

- the DFA states Q' are the power set of Q : $Q' = P(Q)$

$$q_0' = E(q_0)$$

$$F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$$

$$\delta' \neq \delta$$

$$\delta'(R, a) = E\left(\bigcup_{q \in R} (\delta, a)\right)$$

We prove by induction on the length of a string w that the extended transition function of M' on w equals the set of states reachable in M by w :

$$\delta'(q_0', w) = \delta^*(q_0, w)$$

Base case:

$$\delta'(q_0', \epsilon) = \delta^*(q_0, \epsilon)$$

Inductive hypothesis:

Assume, $\delta'(q_0', x) = \delta^*(q_0, x)$ for all strings x of length k . Let $R = \delta^*(q_0, x)$

Inductive Step:

$$w = xa,$$

$$\begin{aligned} \delta'(q_0', xa) &= \delta'(\delta'(q_0', x), a) \\ &= \delta'(R, a) \\ &= \epsilon \left(\bigcup_{q \in R} \delta(q, a) \right) \end{aligned}$$

$$\begin{aligned} q_0 \delta^*(q_0, xa) &= \epsilon \left(\bigcup_{q \in \delta^*(q_0, x)} \delta(q, a) \right) \\ &= \epsilon \left(\bigcup_{q \in R} \delta(q, a) \right) \end{aligned}$$

Since $\delta'(R, q) = \delta^*(q_0, \alpha q)$, the induction ~~holds~~ holds.

Also,

a string w is accepted by M if $\delta^*(q_0, w) \cap F \neq \emptyset$. A string w is accepted by M' if $\delta'(q'_0, w) \in F$, which means $\delta'(q'_0, w) \cap F \neq \emptyset$.

Since $\delta'(q'_0, w) = \delta^*(q_0, w)$, the condition for acceptance is identical for both automata.

thus $L(M) = L(M')$

Hence proved.

DECISION PROPERTIES OF REGULAR LANGUAGES

- (i) Emptiness - is $L(M) = \emptyset$?
- (ii) Finiteness - is $L(M)$ finite?
- (iii) Membership - is $w \in L(M)$?
- (iv) Equivalence - is $L(M_1) = L(M_2)$?
- (v) Subset - is $L(M_1) \subseteq L(M_2)$?
- (vi) Disjointness - is $(L(M_1) \cap L(M_2)) = \emptyset$?
- (vii) Universality - is $L(M) = \Sigma^*$?
- (viii) Equivalence (RE vs PA) - Does $L(R) = L(M)$?

APPLICATIONS OF REGULAR LANGUAGES?

① Lexical Analysis in Compilers:

Regular exp are used to describe the lexical structure (tokens) of programming languages - such as identifiers, keywords, numbers, operators, and punctuation.

② Text searching and Pattern Matching:

Regular expressions are the basis of search commands and utilities such as UNIX - grep, sed, and awk, as well as search functionality in editors like VS Code, sublime, vim.

③ String Validation and Input Filtering:

REs are used to check whether a given string follows a particular format or syntax rule.

④ Network Security and Data Filtering:

REs are used in intrusion detection systems, firewalls, and email filters to identify suspicious patterns (eg. SQL injection, phishing, URLs etc).

⑤ Syntax Specification in Data Formats

REs are used in defining syntactic rules in data description languages like:

- JSON schema
- XML schema (XSD)
- Regular grammars in configuration file validation.

⑥ Digital circuit and Control Design:

REs can model sequences of inputs or control signals in digital systems or communication protocols.

⑦ Natural Language Processing:

Used in NLP processing tasks for tokenization, entity extraction, and simple linguistic pattern recognition.